

Project Description

In this project, you will design and implement a **browser-based control interface** for a physical robot. A user will control the robot from a phone or laptop by connecting directly to the robot's IP address.

The system architecture is:

Web Browser (HTML/JS) → Flask Server (on Pi) → Robot Control Code (Python)

This project emphasizes **human-robot interaction**, **system integration**, and **reliability**, not autonomy.

Required System Capabilities

1. Hardware Bring-Up & Verification (Step A – Required)

Before any web control:

You must demonstrate that the following components work **from Python code**:

- Drive wheels
- Head tilt
- Head pan
- Waist rotation
- Any broken items must be replaced and fixed before going on.

Arms:

- Must be tested and verified
- **Not required** to be controlled via the web interface
- **Need to know what's broken so you can fix later, not for this project.**

Deliverable:

- A short Python script (or module) that individually exercises wheels, head and waist.
- You can use the Maestro Control
- Clear indication of directions and limits

This step exists to **separate hardware problems from software problems**.

2. Robot Control Layer (Python – Required)

You must implement a control layer that:

- Exposes functions for:
 - Driving (forward/back/turn via wheel speeds)
 - Head tilt
 - Head pan
 - Waist rotation
- Enforces **safe limits** on motion
- Has a clearly defined **STOP / neutral** state

This layer:

- Must be independent of Flask
- Must be callable directly from Python for testing

3. Flask Control Server (Required)

A Flask server running on the robot must:

- Serve the HTML/JavaScript page
 - You run `>>python myApp.py`
 - And then use `http://<robot-ip>:YourPortNumber/`
 - And flask will serve the Webpage, Flask will serve as the webserver.
 - (Remember you can use A.I. to show you how flask can host the Webpage.
- Accept control commands from the webpage
- Forward validated commands to the robot control layer

Requirements:

- Flask runs **on the robot**
- HTML/JS files are **hosted by Flask**

- No external cloud services
 - System works entirely on the local network
-

4. Web Control Interface (HTML / JavaScript – Required)

Users must be able to connect by visiting:

http://<robot-ip-address>:<port>

The webpage must provide:

Drive Control

- A joystick (mouse or touch-based)
- Controls left/right wheel motion proportionally

Head Control

- Sliders for:
 - Tilt
 - Pan

Voice Output

- At least **four predefined phrases**
- Triggered by buttons on the webpage

Example phrases (students may customize):

- “Hello, Hunter.”
 - “Hunter is so cool.”
 - “Please do not touch my wheels.”
 - “Hunter is the greatest.”
-

5. Data Flow & Validation (Required)

You must demonstrate and document:

- What data is sent from the browser

- How it is validated on the server
- How it is translated into motor commands

Invalid or unsafe commands must:

- Be rejected
 - Not move the robot
-

6. Failure Awareness & Safety (Required)

Your system must handle at least **three** of the following:

- Page refresh while robot is moving
- Lost network connection
- Invalid joystick values
- Rapid command flooding
- Flask server restart

At minimum:

- Robot must stop safely
 - System must not crash silently
-

Deliverables

1. **LIVE DEMO** during your studio time on **Friday the 13th** (what could go wrong), or **Monday the 16th** for group B. (This will change where Group B will be first next time).
 - Drive robot via joystick
 - Move head and waist via sliders
 - Trigger spoken phrases
2. Demonstrate a failure case

3. Source Code Repository

- Python robot control layer
- Flask server
- HTML / JS files

4. Debug / Reflection Log (ties to 20%, graded separately, everyone turns in one to CANVAS)

- At least 6 dated entries
- Hardware issues vs software issues
- One incorrect assumption discovered

Grading Rubric – Project 1A

Category	%
Hardware bring-up & verification	20
Robot control layer quality	20
Flask server & data handling	20
Web interface design & usability	15
Safety & failure handling	10
Demo quality	10
Debug / reflection log	Graded separately
Total	100